

Experiment NOP on password_check.o with target Target.X86_64

Settings

- **program_file:** /home/dblshot/ghPackages/FaultSim/experiments/para_nop_x86_64_password_check/password_check.o
- **out_dir:** /home/dblshot/ghPackages/FaultSim/experiments/para_nop_x86_64_password_check/mutated_bins
- **program_input:** ['w', 'r', 'o', 'n', 'g', '\n']
- **expected_stdout:** ['C', 'o', 'r', 'r', 'e', 'c', 't']
- **expected_returncode:** 0
- **list_expected:** False
- **timeout:** 4
- **save_results:** /home/dblshot/ghPackages/FaultSim/experiments/para_nop_x86_64_password_check/results.csv
- **yes:** True

Binary information + Running the binary information

- Contains **123** instructions
- Therefore, FaultSim attempted to make **123** mutations
- Of the **123** attempted mutations, **123** valid mutated binaries were generated
- The target arch was Target.X86_64
- The compile command was: gcc experiments/para_nop_x86_64_password_check/password_check.o -o experiments/para_nop_x86_64_password_check/
- An example run command: timeout 4s /home/dblshot/ghPackages/FaultSim/experiments/para_nop_x86_64_password_check/
- The NOP for this binary is: Nop.X86_64
- The runtime to generate and run binaries was: 0:00:00.294750

Return Code Frequencies

<i>Exitcode</i>	<i>Frequency</i>
password denied (97)	91
EX SIGSEGV (139)	21
EX SIGABRT (134)	6
password accepted (0)	3
135	1
6	1

Programs that ran critical code

3 programs ran the critical code out of **123** mutated binaries. The binaires were: - password_check.o_0x12ad - password_check.o_0x12c9 - password_check.o_0x12cd

Program 0 password_check.o_0x12ad diassemblebly vs vanilla

0 0x12a3 48 89 c7	mov rdi, rax	0x12a3 48 89 c7	mov rdi, rax
1 0x12a6 e8 65 fe ff ff	call 0x1110	0x12a6 e8 65 fe ff ff	call 0x1110
2 0x12ab 85 c0	test eax, eax	0x12ab 85 c0	test eax, eax
3 0x12ad 75 1a	jne 0x12c9	0x12ad 90	nop
4 0x12af c6 45 ec 01	mov byte ptr [rbp - 0x14], 1	0x12ae 90	nop
5 0x12b3 eb 14	jmp 0x12c9	0x12af c6 45 ec 01	mov byte ptr [rbp - 0x14], 1
6 0x12b5 48 8d 05 7c 0d 00 00	lea rax, [rip + 0xd7c]	0x12b3 eb 14	jmp 0x12c9
7		0x12b5 48 8d 05 7c 0d 00 00	lea rax, [rip + 0xd7c]

Program 1 password_check.o_0x12c9 diassemblebly vs vanilla

0 0x12bf b8 00 00 00 00	mov eax, 0	0x12bf b8 00 00 00 00	mov eax, 0	
1 0x12c4 e8 17 fe ff ff	call 0x10e0	0x12c4 e8 17 fe ff ff	call 0x10e0	
2 0x12c9 80 7d ec 00	cmp byte ptr [rbp - 0x14], 0	0x12c9 90	nop	
3 0x12cd 74 16	je 0x12e5	0x12ca 90	nop	
4 0x12cf 48 8d 05 6b 0d 00 00	lea rax, [rip + 0xd6b]	0x12cb 90	nop	
5		0x12cc 90	nop	
6		0x12cd 74 16	je 0x12e5	
7		0x12cf 48 8d 05 6b 0d 00 00	lea rax, [rip + 0xd6b]	

Program 2 password_check.o_0x12cd diassemblebly vs vanilla

0 0x12c4 e8 17 fe ff ff	call 0x10e0	0x12c4 e8 17 fe ff ff	call 0x10e0
1 0x12c9 80 7d ec 00	cmp byte ptr [rbp - 0x14], 0	0x12c9 80 7d ec 00	cmp byte ptr [rbp - 0x14], 0
2 0x12cd 74 16	je 0x12e5	0x12cd 90	nop
3 0x12cf 48 8d 05 6b 0d 00 00	lea rax, [rip + 0xd6b]	0x12ce 90	nop
4 0x12d6 48 89 c7	mov rdi, rax	0x12cf 48 8d 05 6b 0d 00 00	lea rax, [rip + 0xd6b]
5		0x12d6 48 89 c7	mov rdi, rax

Source Code Lines

```
#include <stdio.h>
#include <stdlib.h> // Required for the exit() function
#include <stdbool.h>
#include <string.h>
// Below provides "EXIT_SUCCESS"
#include <stdbool.h>

// Correct password:
#define PASSWORD "pass"

// Password input max length:
#define MAX_LENGTH 10

/*
 * int main
 * A sample program to that has a user enter a password and sees if it's correct
 */
int main() {

    // Buffer to store user input (extra byte for null terminator)
    char input[MAX_LENGTH + 1];
    printf("Enter the password (max %d characters): ", MAX_LENGTH);

    bool password_correct = false;

    // fgets takes arguments: (buffer, buffer_size, input)
    // this assures that the input is no longer than the size of the buffer
    if (fgets(input, sizeof(input), stdin) != NULL) {
        // If the entered password exceeds the
        // buffer it's incorrect
        if (strchr(input, '\n') == NULL) {
            password_correct = false;
        }
        else {
            // Remove the newling character
            input[strcspn(input, "\n")] = '\0';

            // NOTICE: fgets and strchr make sure the password
            // is the correct length so strcmp is safe :)
            if (strcmp(input, PASSWORD) == 0) {
                password_correct = true;
            }
        }
    }
    else {
        printf("no input");
    }

    // Compare the input with the predefined password
```

```
if (password_correct == 1) {
    printf("Correct\n");
    // Exit with 0 when exit is correct!
    return EXIT_SUCCESS;
} else {
    printf("Wrong\n");
    return 97;
}

return 84;
}
```